Code Explanation

Creating Sample Data for SQL Generation

The provided code is used to create sample data for testing SQL generation using Apache Spark. It defines a function `create_sample_data` that takes a `SparkSession` object as an argument and creates sample tables with data.

Overview of the Code

The code creates three sample tables: `b_source.ord_hdr`, `b_source.ord_dtl`, and `s_master.sale_orders`. The `b_source.ord_hdr` table contains header information for orders, `b_source.ord_dtl` contains detailed information for orders, and `s_master.sale_orders` is a master table that stores sales order information. The code uses Spark SQL to create and populate these tables.

Step 1: Importing Necessary Modules and Defining the Function

The code starts by importing the necessary `SparkSession` module from `pyspark.sql`. It then defines a function `create_sample_data` that takes a `SparkSession` object as an argument.

```
from pyspark.sql import SparkSession

def create_sample_data(spark):
    """
    Create sample data for testing the SQL generation

    Args:
        spark: SparkSession
    """
```

Step 2: Creating the `b_source.ord_hdr` Table

The function creates a table named `b_source.ord_hdr` with the specified columns using Spark SQL. The `CREATE OR REPLACE TABLE` statement is used to create the table if it does not exist or replace it if it already exists.

```
spark.sql("""
CREATE OR REPLACE TABLE b_source.ord_hdr (
    referenceto STRING,
    refno STRING,
    stndard STRING,
    custhead STRING,
    custdtl STRING,
    cusnum STRING,
    cusa STRING,
    ordsthdr STRING,
    osrc STRING,
    cref STRING,
    reftp STRING,
    refntp STRING,
    refnp STRING,
```

```
    reftpo STRING,
    curchtyp STRING,
    dateupd STRING,
    cusn STRING,
    enitycode STRING,
    delete_flag STRING
)
""")
```

Step 3: Inserting Data into the `b_source.ord_hdr` Table

The code inserts sample data into the `b_source.ord_hdr` table using the `INSERT INTO` statement.

```
spark.sql("""
INSERT INTO b_source.ord_hdr VALUES
('REF001', '12345', 'STD', 'CUST1', 'DT1', 'CUS1', 'A', 'ACTIVE', 'P
O', 'CREF1', 'RT1', 'RN1', 'NA', 'RTO1', 'USD', '20230101', 'CN1', '
ENT1', ''),
('REF002', '67890', 'STD', 'CUST2', 'DT2', 'CUS2', 'B', 'PENDING', '
SO', 'CREF2', 'RT2', 'RN2', 'NA', 'RTO2', 'EUR', '20230201', 'CN2',
'ENT2', ''),
('REF003', '13579', 'STD', 'CUST3', 'DT3', 'CUS3', 'C', 'CLOSED', 'P
O', 'CREF3', 'RT3', 'RN3', 'NA', 'RTO3', 'GBP', '20230301', 'CN3', '
ENT3', 'D')
""")
```

Step 4: Creating the `b_source.ord_dtl` Table

The function creates another table named `b_source.ord_dtl` with the specified columns using Spark SQL.

```
spark.sql("""
CREATE OR REPLACE TABLE b_source.ord_dtl (
    referenceto STRING,
    refno STRING,
    bdcob STRING,
    seqnumber STRING,
    prodmatnum STRING,
    ordstline STRING,
    ordq DECIMAL(17,4),
    tcqt DECIMAL(17,4),
    tdqt DECIMAL(17,4),
    totalpric DECIMAL(30,12),
    upbc DECIMAL(30,12),
    datecr STRING,
    dtdel STRING,
    exttx STRING,
    seqne STRING,
    referenceno STRING,
    reqno STRING,
    exttax STRING,
    delete_flag STRING
)
```

```
""")
```

Step 5: Inserting Data into the `b_source.ord_dtl` Table

The code inserts sample data into the `b_source.ord_dtl` table.

```
spark.sql("""
INSERT INTO b_source.ord_dtl VALUES
('REF001', '12345', 'COMP1', '1', 'MAT001', 'SHIPPED', 10.0000, 0.00
00, 10.0000, 100.000000000000, 10.000000000000, '20230110', '2023012
0', 'EXT1', '1', 'REFNO1', 'REQ1', 'EXTAX1', ''),
('REF002', '67890', 'COMP2', '1', 'MAT002', 'PENDING', 5.0000, 0.000
0, 0.0000, 50.000000000000, 10.000000000000, '20230210', '20230220',
 'EXT2', '1', 'REFNO2', 'REQ2', 'EXTAX2', ''),
('REF003', '13579', 'COMP3', '1', 'MAT003', 'CANCELLED', 8.0000, 8.0
000, 0.0000, 80.000000000000, 10.000000000000, '20230310', '20230320
', 'EXT3', '1', 'REFNO3', 'REQ3', 'EXTAX3', 'D')
""")
```

Step 6: Creating the `s_master.sale_orders` Table

The function creates a table named `s_master.sale_orders` with the specified columns.

```
spark.sql("""
CREATE OR REPLACE TABLE s_master.sale_orders (
    OrderNumber STRING,
    OrderType STRING,
    SalesOrgCompanyCode STRING,
    LineNumber STRING,
    ShipToNumber STRING,
    SoldToNumber STRING,
    MaterialNumber STRING,
    OrderStatusLineLast STRING,
    OrderStatusHeader STRING,
    PoDocType STRING,
    CustomerPo STRING,
    RelatedOrderNumber STRING,
    RelatedOrderType STRING,
    OrderQuantityOriginal DECIMAL(17,4),
    OrderQuantityBase DECIMAL(17,4),
    CancelledQuantityOriginal DECIMAL(17,4),
    DeliveredQuantityOriginal DECIMAL(17,4),
    OpenQuantityOrginal DECIMAL(17,4),
    OpenQuantityBase DECIMAL(17,4),
    CurrencyType STRING,
    TotalPriceLocal DECIMAL(30,12),
    UnitPriceLocal DECIMAL(30,12)
)
""")
```